

DEVELOPER GUIDE · VERSION 1.0

eCatalogus Public API v1

Reading and importing liturgical manuscript data: authentication, endpoints, step-by-step use cases and real responses.

Interactive reference (Swagger): <https://<instance>/api/schema/swagger/>

OpenAPI schema: <https://<instance>/api/schema/>

API root: <https://<instance>/api/v1/>

Instytut Sztuki Polskiej Akademii Nauk (PAN)

Data licence: Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)

Document date: 28 September 2026. Read examples are real responses from the public instances; write examples come from the same code running against a test database.

Contents

1. Overview

1. Instances and base URLs
2. Endpoints at a glance

2. Getting access

1. Reading: no key needed
2. Writing: how to obtain credentials
3. Verifying credentials
4. Calling from a browser (CORS)

3. Conventions

1. Identifiers
2. Pagination
3. Labels
4. Errors
5. Rate limits and sizes
6. Licence and citation

4. Endpoint reference

5. Use cases, step by step

1. Find a manuscript and read its content
2. Export a manuscript's content to CSV
3. Mirror a vocabulary and keep it in sync
4. Resolve identifiers you already hold
5. Import a manuscript with its content
6. Correct and re-upload content
7. Upload from a web page (JavaScript)
8. Cite the data you used

6. Field reference

1. Content row
2. Manuscript
3. Vocabularies

7. Error reference

8. Troubleshooting and FAQ

9. Appendix: a small Python client

1. Overview

eCatalogus is a catalogue of medieval liturgical manuscripts maintained by the Institute of Art of the Polish Academy of Sciences. Its **Public API v1** lets other systems:

- **read** manuscripts, their complete descriptions and their content (the liturgical items — prayers, chants, readings — in manuscript order);
- **read** the controlled vocabularies (rubrics, formulas, liturgical functions, places, datings...) with stable UUIDs;
- **create** manuscripts and **bulk-import** content, with full validation and all-or-nothing writes.

The API speaks JSON over HTTPS. Reading is public and needs no key. Writing needs an eCatalogus user account with import rights (section 2).

1.1 Instances and base URLs

eCatalogus runs as several instances, each with its own manuscripts. They all serve the same API. The controlled vocabularies are curated on the main eCatalogus instance and replicated to all the others, so **a vocabulary UUID is the same everywhere**. A manuscript lives on one instance, and its UUID is valid there.

Instance	API base URL
eCatalogus	https://ecatalogus.ispan.pl/api/v1/
MPL Limbo	https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/
Liturgica Poloniae	https://monumenta-poloniae-liturgica.ispan.pl/api/v1/
Canon Missae	https://canon-missae.ispan.pl/api/v1/
Corpus Liturgicum	https://corpus-liturgicum.org/api/v1/

The vocabularies are curated on eCatalogus, which is the master copy. To see what an instance holds, call its `/manuscripts/` and `/dictionaries/` endpoints.

The examples in this guide use MPL Limbo for manuscript data and eCatalogus for vocabularies. Replace the host to work with another instance.

Your first call

```
curl https://ecatalogus.ispan.pl/api/v1/
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "documentation": "https://ecatalogus.ispan.pl/api/schema/swagger/",
  "endpoints": {
    "whoami": "https://ecatalogus.ispan.pl/api/v1/whoami/",
    "manuscripts": "https://ecatalogus.ispan.pl/api/v1/manuscripts/",
    "manuscript_package": "https://ecatalogus.ispan.pl/api/v1/manuscripts/{uuid}/package/",
    "manuscript_content": "https://ecatalogus.ispan.pl/api/v1/manuscripts/{uuid}/content/",
    "manuscript_content_summary":
```

```

"https://ecatalogus.ispan.pl/api/v1/manuscripts/{uuid}/content/summary/",
  "manuscript_content_bulk": "https://ecatalogus.ispan.pl/api/v1/manuscripts/{uuid}/content/bulk/",
  "dictionaries": "https://ecatalogus.ispan.pl/api/v1/dictionaries/"
},
"rights": {
  "id": "CC-BY-NC-4.0",
  "name": "Creative Commons Attribution-NonCommercial 4.0 International",
  "url": "https://creativecommons.org/licenses/by-nc/4.0/",
  "copyright": "Copyright (c) 2024–2026 Instytut Sztuki Polskiej Akademii Nauk (PAN) – Polish Academy of Sciences.",
  "rights_holder": "Instytut Sztuki Polskiej Akademii Nauk (PAN)",
  "rights_holder_url": "https://ispan.pl/",
  "required_statement": "Data from eCatalogus, Instytut Sztuki Polskiej Akademii Nauk (PAN). Used under CC BY-NC 4.0; not for commercial use."
}
}

```

The `endpoints` map holds URL templates: replace `{uuid}` with a manuscript UUID.

1.2 Endpoints at a glance

Method and path	Purpose	Access
GET <code>/api/v1/</code>	Discovery document	public
GET <code>/api/v1/whoami/</code>	Check whose credentials were sent, and whether they may import	public
GET <code>/api/v1/manuscripts/</code>	List and search manuscripts	public
POST <code>/api/v1/manuscripts/</code>	Create a manuscript	importer
GET <code>/api/v1/manuscripts/{uuid}/package/</code>	Complete description of one manuscript	public
GET <code>/api/v1/manuscripts/{uuid}/content/</code>	Content rows of one manuscript	public
GET <code>/api/v1/manuscripts/{uuid}/content/summary/</code>	Does the manuscript already have content?	public
POST <code>/api/v1/manuscripts/{uuid}/content/bulk/</code>	Bulk-import content rows	importer
GET <code>/api/v1/dictionaries/</code>	List the controlled vocabularies	public
GET <code>/api/v1/dictionaries/{slug}/</code>	Read one vocabulary	public

Other paths exist on the server: `/api/etl/` is internal replication between instances, and `/api/content/`, `/api/manuscripts/` and similar serve the site's own pages. They are not part of the public API, and they may change without notice.

2. Getting access

2.1 Reading: no key needed

Every `GET` endpoint is public. There is no API key, no registration and no token. Just call it:

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/?search=Tinecense'
```

Anonymous traffic is limited to 15 requests per minute. For larger harvests, sign in with any eCatalogus account, which lifts the limit completely. Section 3.5 has the details.

2.2 Writing: how to obtain credentials

Creating manuscripts and importing content requires an ordinary **eCatalogus user account** (username and password) that has import rights. The API has **no separate API keys or tokens**: the same account a person uses to sign in to the website is sent with each write request as HTTP Basic authentication over HTTPS.

An account may write through the API when any one of these is true:

- it is a member of the `api_importers` group (the usual way);
- it is a superuser;
- it holds the Django model permission for the table being written (`indexerapp.add_manuscripts` or `indexerapp.add_content`). Existing editorial accounts usually already have this.

Step by step: getting import rights

1 Get an account on the instance you will write to

Each instance has its own user accounts. Register on the instance's website, or ask its administrator to create an account for you. For an unattended integration (a server importing on a schedule), ask for a dedicated account instead of using a person's.

2 Ask the administrator to grant import rights

Send the administrator your username and the instance name. The administrator does one of the following:

In the Django admin: Authentication and Authorization → Users → (*your user*) → Groups → add `api_importers` → Save.

Or on the server:

```
# grant import rights to an existing account (nothing else about it changes)
python manage.py create_api_integration_user akowalska

# create a dedicated unattended account; a password is generated and printed if omitted
python manage.py create_api_integration_user ritus-bot --password 'a-long-random-password'

# revoke
python manage.py create_api_integration_user akowalska --remove
```

The change takes effect immediately and needs no restart. An account created by the command is deliberately not staff and not a superuser: it can import through the API and do nothing else.

3

Verify

Call `/api/v1/whoami/` with your credentials (next section). `"can_import": true` means you are ready.

Keep credentials out of code and storage. For a person using a web tool, ask for the password at upload time and keep it in memory only. For a server integration, read it from an environment variable or a secret store. Always use `https://`: Basic authentication is readable over plain HTTP.

2.3 Verifying credentials

```
curl -u akowalska:PASSWORD https://ecatalogus.ispan.pl/api/v1/whoami/
```

RESPONSE **HTTP 200** *valid account in api_importers*

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "authenticated": true,
  "username": "akowalska",
  "display_name": "Anna Kowalska",
  "can_import": true
}
```

Three outcomes, each needing a different message to the user:

Response	Meaning	What to do
401	Wrong username or password	Ask the user to re-enter them
200 , can_import: false	Credentials are fine, but the account has no import rights	Ask the administrator to add the account to <code>api_importers</code>
200 , can_import: true	Ready	Proceed with the upload

RESPONSE **HTTP 200** *valid account without import rights*

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "authenticated": true,
  "username": "gosc",
  "display_name": "gosc",
  "can_import": false
}
```

RESPONSE **HTTP 401** *wrong password*

```
{
  "detail": "Invalid username/password."
}
```

RESPONSE **HTTP 200** *no credentials sent*

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "authenticated": false,
  "username": null,
  "display_name": null,
  "can_import": false
}
```

A failed login answers with the header `WWW-Authenticate: xBasic realm="api"`. Browsers do not recognise the `xBasic` scheme, so a web page calling the API never gets the browser's built-in password pop-up on top of its own login form. Command-line tools are unaffected.

2.4 Calling from a browser (CORS)

JavaScript on another website may call the API only if its origin is on the instance's allow-list. `https://ritus-indexer.ispan.pl` is on the list on every instance. To add your origin, ask the administrator to include it in the `API_INTEGRATION_ORIGINS` setting (a comma-separated list) and restart the instance. The pre-flight response for an allowed origin looks like this:

```
curl -i -X OPTIONS https://canon-missae.ispan.pl/api/v1/manuscripts/{uuid}/content/bulk/ \
  -H "Origin: https://ritus-indexer.ispan.pl" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: authorization,content-type"
```

RESPONSE *headers*

```
HTTP/2 200
server: nginx
date: Mon, 28 Sep 2026 11:57:02 GMT
content-type: text/html; charset=utf-8
```

```
content-length: 0
vary: origin
access-control-allow-origin: https://ritus-indexer.ispan.pl
access-control-allow-credentials: true
access-control-allow-headers: accept, authorization, content-type, user-agent, x-csrftoken, x-requested-with
access-control-allow-methods: DELETE, GET, OPTIONS, PATCH, POST, PUT
access-control-max-age: 86400
x-content-type-options: nosniff
referrer-policy: same-origin
cross-origin-opener-policy: same-origin
```

Send credentials in the `Authorization` header. Do **not** set `credentials: 'include'`, because the API needs no cookies. Public reads work from any origin that is on the allow-list.

3. Conventions

3.1 Identifiers

- Every record is identified by a **UUID**. Store UUIDs, never numeric ids.
- Numeric database ids are different on every instance and are **not published**. There are two exceptions: `rite-names` and `formulas` publish `id`, which is kept identical on all instances.
- Vocabulary entries carried over from the legacy database have UUIDs derived from their old numeric id. Section 5.4 shows how to resolve them with `?legacy_ids=`.
- Manuscripts may carry `foreign_id`, the identifier an external system gave them. Filter on it with `?foreign_id=`.

3.2 Pagination

List endpoints take `limit` and `offset` and return `count`, `limit`, `offset` and `next_offset`. Keep requesting with `offset=next_offset` until `next_offset` is `null`.

Endpoint	Default page	Maximum
manuscripts, content	100	2000
dictionaries/{slug}	200 (1000 when selecting by <code>uuids</code> or <code>legacy_ids</code>)	1000

Values above the maximum are clamped, not rejected. The applied value is echoed in `limit`.

3.3 Labels

A relation is returned twice: its UUID, and a human-readable `*_label` next to it. You can display rows without downloading any vocabulary:

```
"function_id": "85d62046-5521-5150-aa23-799de39d44f9",  
"function_label": "Collecta",
```

When writing, send either the UUID or the name, as section 6.1 explains. Labels you send back are ignored.

3.4 Errors

Every error is JSON with a `detail` message. Validation failures on the write endpoints add an `errors` list that names the row, field and value of **every** problem, not just the first one. Section 7 lists all codes.

RESPONSE **HTTP 404** GET .../manuscripts/00000000-0000-0000-0000-000000000000/content/summary/

```
{  
  "detail": "No manuscript with uuid 00000000-0000-0000-0000-000000000000."  
}
```

3.5 Rate limits and sizes

- **Anonymous requests:** 15 per minute per IP address. Beyond that the server answers `429 Too Many Requests` with a `Retry-After` header saying how many seconds to wait. Authenticated requests are never throttled, and any eCatalogus account will do, with or without import rights. Send credentials (`curl -u user:password`) for anything beyond occasional lookups.
- **Bulk import:** no row limit and no rate limit. A request body may be up to 256 MB. Allow about one second per hundred rows, and set a generous client timeout.

3.6 Licence and citation

The data is published under **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)**. You may copy, adapt and redistribute it for **non-commercial** purposes, such as research, teaching and non-profit projects, provided you credit it. Commercial use requires separate permission from the rights holder, the Institute of Art of the Polish Academy of Sciences. Every response carries the licence in a header:

```
Link: <https://creativecommons.org/licenses/by-nc/4.0/>; rel="license"
```

Every read response also contains a `rights` block. The block is omitted from most examples in this guide to save space. It contains the attribution you are required to show and a ready-made citation. Section 5.8 shows how to use them.

4. Endpoint reference

Paths are relative to the instance's API root, e.g. `https://limbo.monumenta-poloniae-liturgica.ispan.pl`. All responses are `application/json`.

GET `/api/v1/whoami/`

public · never throttled

Returns the identity behind the request and whether it may import. Section 2.3 has all four responses.

GET `/api/v1/manuscripts/`

public

Manuscripts on this instance, ordered by name, each with its number of content rows.

Parameter	Description
<code>search</code>	Case-insensitive substring of the name, shelf mark, RISM id or foreign id
<code>foreign_id</code>	Exact match on the external system's identifier
<code>limit</code> , <code>offset</code>	Paging (default 100, max 2000)

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/?search=Tinecense'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "MPL Limbo",
  "count": 1,
  "limit": 100,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "7c0a82e8-8a59-5362-95d3-5d70e879dbf8",
      "name": "Sacramentarium Tinecense",
      "rism_id": "PL-Wn B0Z 8",
      "foreign_id": "127",
      "shelf_mark": "Rps B0Z 8",
      "contemporary_repository_place_label": "The National Library",
      "dating_label": "XI med",
      "content_count": 1416,
      "entry_date": "2025-12-10T20:00:26.921052+00:00"
    }
  ]
}
```

Creates one manuscript and returns its `uuid`. Only `name` is required. Relation fields accept a UUID or a vocabulary name (section 6.2). Unknown names are rejected, because the API never creates vocabulary entries. Unrecognised keys are rejected too, unless you pass `?strict=false`.

```
curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/ \
-H 'Content-Type: application/json' \
-d '{
  "name": "Graduale Cracoviense",
  "foreign_id": "ritus-2291",
  "rism_id": "PL-Kk 12",
  "shelf_mark": "MS 12",
  "contemporary_repository_place": "Biblioteka Narodowa",
  "dating": "XIV",
  "main_script": "Textualis"
}'
```

RESPONSE HTTP 201

```
{
  "uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "name": "Graduale Cracoviense",
  "foreign_id": "ritus-2291",
  "content_count": 0,
  "entry_date": "2026-09-28T12:13:36.191153+00:00"
}
```

A request with problems is rejected as a whole, with every problem listed:

```
-d '{"name": "Graduale Cracoviense", "dating": "s. XXX", "shelfmark": "MS 12"}'
```

RESPONSE HTTP 400

```
{
  "detail": "2 problem(s) found; the manuscript was not created.",
  "errors": [
    {
      "row": 0,
      "field": "shelfmark",
      "value": "MS 12",
      "error": "unknown_field",
      "detail": "\"shelfmark\" is not a recognised manuscript field."
    },
    {
      "row": 0,
      "field": "dating",
      "value": "s. XXX",
      "error": "not_found",

```

```

    "detail": "No TimeReference entry matches \"s. XXX\" (matched against: time_description).
    Dictionary entries must exist before content referencing them is imported."
  }
}
}

```

GET `/api/v1/manuscripts/{uuid}/package/`

public

Everything attached to one manuscript: the manuscript record, codicology, layouts, quires, binding, decoration, origins, provenance, hands, watermarks, projects, images and content. It is the complete data behind the manuscript page, and a superset of the TEI export. The response has one block per model. Each foreign key `*_uuid` is followed by its `*_label`. Pass `?labels=false` for UUIDs only.

Images are not embedded. `media_files` lists each attached image with its `path` (the value the records refer to), its `size` in bytes and an absolute `url`. Download only the images you need, straight from the web server.

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/28d56828-30eb-45fe-9034-d3d7d6e88d12/package/'
```

RESPONSE **HTTP 200** *null and empty fields removed for brevity*

```

{
  "site_name": "MPL Limbo",
  "category": "ms",
  "manuscript_uuid": "28d56828-30eb-45fe-9034-d3d7d6e88d12",
  "manuscript_name": "Antiphonarium",
  "model_count": 2,
  "record_count": 2,
  "models": [
    {
      "model": "indexerapp.Manuscripts",
      "category": "ms",
      "count": 1,
      "results": [
        {
          "uuid": "28d56828-30eb-45fe-9034-d3d7d6e88d12",
          "name": "Antiphonarium",
          "rism_id": "PL-WRu I F 405",
          "foreign_id": "MSPL 5197",
          "contemporary_repository_place_uuid": "eb09cafc-bde0-5f5a-89b7-014fd6a47eae",
          "shelf_mark": "I F 405",
          "dating_uuid": "93b1b867-a1b7-5d66-a378-1ce3bca34624",
          "links": "https://glam.uni.wroc.pl/index.php?s=RKP_I_F_405_5255&view=thumbnails&thumb_size=medium#current",
          "image": "images/Zrzut_ekranu_2026-07-1_o_10.22.40.png",
          "sync_status": "in_preparation",
          "entry_date": "2026-07-01T08:26:44.682389+00:00",
          "data_contributor_uuid": "f34f11c0-9d7d-5041-ba01-cd9c99e3a167",
          "contemporary_repository_place_label": "University Library",
          "dating_label": "XIII",

```

```

        "data_contributor_label": "PPŻ"
      }
    ]
  },
  {
    "model": "indexerapp.MSProjects",
    "category": "ms",
    "count": 1,
    "results": [
      {
        "uuid": "4497f26e-5d6b-485a-8e0a-e46cf96a8d86",
        "manuscript_uuid": "28d56828-30eb-45fe-9034-d3d7d6e88d12",
        "project_uuid": "7762c02f-8d5c-5bde-8924-8e6fa3ee2650",
        "entry_date": "2026-06-02T09:22:09.818011+00:00",
        "manuscript_label": "Antiphonarium",
        "project_label": "Liturgica Poloniae"
      }
    ]
  }
],
"media_files": [
  {
    "path": "images/Zrzut_ekranu_2026-07-1_o_10.22.40.png",
    "size": 3652450,
    "url": "https://limbo.monumenta-poloniae-liturgica.ispan.pl/media/images/Zrzut_ekranu_2026-07-1_o_10.22.40.png"
  }
],
"api_version": "v1"
}

```

GET `/api/v1/manuscripts/{uuid}/content/`

public

The manuscript's content rows in manuscript order, in exactly the format the bulk import accepts. Paging: `limit`, `offset` (default 100, max 2000). Every row always carries every key listed in section 6.1, with `null` where there is no value.

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/7c0a82e8-8a59-5362-95d3-5d70e879dbf8/content/?limit=1&offset=10'
```

RESPONSE **HTTP 200** *complete, untrimmed row*

```

{
  "api_version": "v1",
  "manuscript_uuid": "7c0a82e8-8a59-5362-95d3-5d70e879dbf8",
  "manuscript_name": "Sacramentarium Tinecense",
  "count": 1416,
  "limit": 1,
  "offset": 10,
  "next_offset": 11,

```

```

"results": [
  {
    "uuid": "11b726a0-6ac5-5478-8aa4-572490629e8c",
    "manuscript_uuid": "7c0a82e8-8a59-5362-95d3-5d70e879dbf8",
    "sequence_in_ms": 11,
    "rubric_sequence_in_the_MS": null,
    "rubric_name_from_ms": "Missa pro rege et exercitu eius",
    "subrubric_name_from_ms": null,
    "formula_text_from_ms": null,
    "where_in_ms_from": "5",
    "where_in_ms_to": "6",
    "digital_page_number": null,
    "original_or_added": "ADDED",
    "biblical_reference": null,
    "reference_to_other_items": null,
    "similarity_by_user": "0",
    "edition_subindex": null,
    "comments": null,
    "proper_texts": null,
    "formula_id": "2f1a32fb-721b-5655-9865-1b10eea70a10",
    "formula_label": "2046",
    "rubric_id": null,
    "rubric_label": null,
    "liturgical_genre_id": null,
    "liturgical_genre_label": null,
    "quire_id": null,
    "quire_label": null,
    "section_id": null,
    "section_label": null,
    "subsection_id": null,
    "subsection_label": null,
    "function_id": "85d62046-5521-5150-aa23-799de39d44f9",
    "function_label": "Collecta",
    "subfunction_id": null,
    "subfunction_label": null,
    "music_notation_id": null,
    "music_notation_label": null,
    "layer": null,
    "layer_label": null,
    "mass_hour": null,
    "mass_hour_label": null,
    "genre": null,
    "genre_label": null,
    "season_month": null,
    "season_month_label": null,
    "week": null,
    "week_label": null,
    "day": null,
    "day_label": null,
    "text_standarization": null,
  }
]

```

```

    "text_standarization_label": null,
    "contributor_id": "86c8f363-c223-5a50-89ba-b57d72dad5cf",
    "contributor_label": "PF",
    "edition_index": null,
    "edition_index_label": null,
    "entry_date": "2024-12-04T20:04:14.824768+00:00"
  }
]
}

```

GET `/api/v1/manuscripts/{uuid}/content/summary/`

public

A cheap probe. Call it before uploading, so that a retry does not duplicate content.

```
curl https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/7c0a82e8-8a59-5362-95d3-5d70e879dbf8/content/summary/
```

RESPONSE **HTTP 200**

```

{
  "api_version": "v1",
  "site_name": "MPL Limbo",
  "manuscript_uuid": "7c0a82e8-8a59-5362-95d3-5d70e879dbf8",
  "manuscript_name": "Sacramentarium Tinecense",
  "content_count": 1416,
  "has_content": true,
  "sequence_in_ms_min": 1,
  "sequence_in_ms_max": 1416,
  "last_modified": "2024-12-04T20:04:18.805394+00:00"
}

```

POST `/api/v1/manuscripts/{uuid}/content/bulk/`

importer · no row or rate limit

Imports many content rows in one request. The body is an object, or a bare JSON list of rows (the legacy shape):

Key	Default	Meaning
<code>items</code>	<i>required</i>	List of content rows (section 6.1)
<code>mode</code>	<code>append</code>	<code>append</code> adds rows. <code>replace</code> first deletes all of the manuscript's content, in the same transaction.
<code>dry_run</code>	<code>false</code>	Validate everything and report every problem, but write nothing
<code>strict</code>	<code>true</code>	Reject unrecognised keys. <code>false</code> ignores them.

All or nothing: the whole payload is validated before anything is written, and the write is a single transaction. If any row is bad, nothing is imported. Rows without `sequence_in_ms` are numbered after the highest existing one. Section 5.5 walks through a complete import with real responses.

GET `/api/v1/dictionaries/`

public

The published vocabularies with their current entry counts. Section 6.3 has the full list.


```
curl https://ecatalogus.ispan.pl/api/v1/dictionaries/
```

RESPONSE **HTTP 200** *first 3 entries*

```
{
  "api_version": "v1",
  "count": 32,
  "results": [
    {
      "slug": "binding-components",
      "model": "indexerapp.BindingComponents",
      "verbose_name": "Binding Components",
      "count": 3,
      "url": "https://ecatalogus.ispan.pl/api/v1/dictionaries/binding-components/"
    },
    {
      "slug": "binding-decoration-types",
      "model": "indexerapp.BindingDecorationTypes",
      "verbose_name": "Binding Decoration Types",
      "count": 7,
      "url": "https://ecatalogus.ispan.pl/api/v1/dictionaries/binding-decoration-types/"
    },
    {
      "slug": "binding-materials",
      "model": "indexerapp.BindingMaterials",
      "verbose_name": "Binding Materials",
      "count": 3,
      "url": "https://ecatalogus.ispan.pl/api/v1/dictionaries/binding-materials/"
    }
  ]
}
```

GET `/api/v1/dictionaries/{slug}/`

public

Entries of one vocabulary, oldest first.

Parameter	Description
<code>search</code>	Case-insensitive substring match on the vocabulary's name columns (section 6.3)
<code>since</code>	ISO-8601 timestamp. Returns only entries changed since then. Vocabularies without an <code>entry_date</code> ignore it.
<code>uuids</code>	Comma-separated UUIDs, at most 1000
<code>legacy_ids</code>	Comma-separated legacy numeric ids, at most 1000. Adds <code>legacy_id</code> to each row and <code>unresolved_legacy_ids</code> to the response.
<code>fields</code>	Comma-separated columns to return. <code>uuid</code> is always included, and <code>id</code> too for rite-names and formulas.
<code>limit</code> , <code>offset</code>	Paging (default 200, max 1000)

Any other query parameter is rejected with **400**, so a typo never silently returns the whole vocabulary.

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/time-reference/?search=XIV&limit=3'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "slug": "time-reference",
  "model": "indexerapp.TimeReference",
  "count": 16,
  "limit": 3,
  "offset": 0,
  "next_offset": 3,
  "results": [
    {
      "uuid": "ae18c40d-d0fb-5c67-b67b-bf4d6819ce59",
      "time_description": "XIII/XIV",
      "century_from": 13,
      "century_to": 14,
      "year_from": 1290,
      "year_to": 1310,
      "entry_date": "2026-04-27T09:21:06.012336+00:00"
    },
    {
      "uuid": "f0c88098-b147-5e6b-aef9-5e9bfd0fee07",
      "time_description": "XIV",
      "century_from": 14,
      "century_to": 14,
      "year_from": 1301,
      "year_to": 1400,
      "entry_date": "2026-04-27T09:21:06.012336+00:00"
    },
    {
      "uuid": "fcb0c732-ba89-5a4b-9b78-975ed40e15d1",
      "time_description": "XIV 1/2",
      "century_from": 14,
      "century_to": 14,
      "year_from": 1301,
      "year_to": 1350,
      "entry_date": "2026-04-27T09:21:06.012336+00:00"
    }
  ]
}
```

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/liturgical-genres/?fields=title'
```

RESPONSE **HTTP 200**

```

{
  "api_version": "v1",
  "slug": "liturgical-genres",
  "model": "indexerapp.LiturgicalGenres",
  "count": 9,
  "limit": 200,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "26bca9f2-8994-51d6-9293-7bd74492a860",
      "title": "Sacramentary"
    },
    {
      "uuid": "c4e84f91-2cdc-5875-8d00-d223b457ad09",
      "title": "Missal"
    },
    {
      "uuid": "127fe605-1330-5740-83e2-d0ce5bb763e0",
      "title": "Pontifical"
    },
    {
      "uuid": "153b625a-3a18-5b4c-b143-93f344d198e0",
      "title": "Collectar"
    },
    {
      "uuid": "a40ec15b-0645-54a4-8568-5aaea4c027c9",
      "title": "Calendar"
    },
    {
      "uuid": "c53cee1e-0420-5b43-8a1e-ddf612acd2e8",
      "title": "Ritual"
    },
    {
      "uuid": "c70a7dfd-0a7f-5a2b-8b44-c8170a269ed1",
      "title": "Benedictional"
    },
    {
      "uuid": "d3c281a6-dfd9-5923-9817-fdc6f38ab3ec",
      "title": "Graduale"
    },
    {
      "uuid": "dcc1cd48-f967-5964-9059-db5688eb7198",
      "title": "Evangelistarium"
    }
  ]
}

```

5. Use cases, step by step

5.1 Find a manuscript and read its content

Goal: read the content of the *Sacramentarium Tinecense*. No credentials are needed.

1 Search for the manuscript

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/?search=Tinecense'
```

Take `uuid` from the result: `7c0a82e8-8a59-5362-95d3-5d70e879dbf8`. The response is in the endpoint reference above.

2 See how much content there is

```
curl https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/7c0a82e8-8a59-5362-95d3-5d70e879dbf8/content/summary/
```

The response had `"content_count": 1416` when this guide was written. At 1000 rows per page, that is two pages.

3 Fetch the rows, page by page

```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/7c0a82e8-8a59-5362-95d3-5d70e879dbf8/content/?limit=1000&offset=0' # next_offset: 1000
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/7c0a82e8-8a59-5362-95d3-5d70e879dbf8/content/?limit=1000&offset=1000' # next_offset: null → done
```

Each row carries the folio (`where_in_ms_from` / `_to`), the rubric as written in the manuscript, the standardised formula (`formula_label` is its Corpus Oratorium number) and the liturgical function:

RESPONSE *one row, null keys removed*

```
{
  "uuid": "11b726a0-6ac5-5478-8aa4-572490629e8c",
  "manuscript_uuid": "7c0a82e8-8a59-5362-95d3-5d70e879dbf8",
  "sequence_in_ms": 11,
  "rubric_name_from_ms": "Missa pro rege et exercitu eius",
  "where_in_ms_from": "5",
  "where_in_ms_to": "6",
  "original_or_added": "ADDED",
  "similarity_by_user": "0",
  "formula_id": "2f1a32fb-721b-5655-9865-1b10eea70a10",
  "formula_label": "2046",
  "function_id": "85d62046-5521-5150-aa23-799de39d44f9",
  "function_label": "Collecta",
  "contributor_id": "86c8f363-c223-5a50-89ba-b57d72dad5cf",
  "contributor_label": "PF",
  "entry_date": "2024-12-04T20:04:14.824768+00:00"
}
```

4

Look up the full text of a formula

`formula_label` gives the number only. For the text, look the UUID up in the `formulas` vocabulary:

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/formulas/?uuids=2f1a32fb-721b-5655-9865-1b10eea70a10&fields=co_no,text'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "slug": "formulas",
  "model": "indexerapp.Formulas",
  "count": 1,
  "limit": 1000,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "2f1a32fb-721b-5655-9865-1b10eea70a10",
      "id": 7543,
      "co_no": "2046",
      "text": "Deus qui regnorum omnium regumque es dominator supplicationes nostras clementer exaudi et famuli tui illius re..."
    }
  ]
}
```

5.2 Export a manuscript's content to CSV

This Python script pages through the content and writes the rows to a spreadsheet-friendly CSV, with labels rather than UUIDs. It needs only the `requests` package.

```
import csv
import requests

BASE = "https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1"
MS = "7c0a82e8-8a59-5362-95d3-5d70e879dbf8" # Sacramentarium Tinecense
COLUMNS = ["sequence_in_ms", "where_in_ms_from", "where_in_ms_to",
            "rubric_name_from_ms", "formula_label", "function_label",
            "original_or_added"]

def rows(manuscript_uuid):
    offset = 0
    while offset is not None:
        page = requests.get(f'{BASE}/manuscripts/{manuscript_uuid}/content/',
                           params={"limit": 1000, "offset": offset}, timeout=120).json()
        yield from page["results"]
        offset = page["next_offset"]
```

```

print(page["rights"]["attribution"])          # show the required credit

with open("tinecense.csv", "w", newline="", encoding="utf-8") as handle:
    writer = csv.DictWriter(handle, fieldnames=COLUMNS, extrasaction="ignore")
    writer.writeheader()
    for row in rows(MS):
        writer.writerow(row)

```

OUTPUT (TINECENSE.CSV, ROWS 11–13)

```

sequence_in_ms,where_in_ms_from,where_in_ms_to,rubric_name_from_ms,formula_label,function_label,original_or_added
11,5,6,Missa pro rege et exercitu eius,2046,Collecta,ADDED
12,6,6,Missa pro rege et exercitu eius,5748b,Secreta,ADDED
13,6,6,Missa pro rege et exercitu eius,1160,Ad complendum,ADDED

```

5.3 Mirror a vocabulary and keep it in sync

Goal: keep a local copy of the rite names up to date. A full download takes several requests, so authenticate to avoid the anonymous rate limit.

1 Full download, once

```

curl -u USER:PASSWORD 'https://ecatalogus.ispan.pl/api/v1/dictionaries/rite-names/?
limit=1000&offset=0'
curl -u USER:PASSWORD 'https://ecatalogus.ispan.pl/api/v1/dictionaries/rite-names/?
limit=1000&offset=1000'
# ... until next_offset is null. Record the time you started: T0

```

2 Incremental updates

Next time, ask only for entries changed since T0, and upsert them by `uuid` :

```

curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/rite-names/?since=2026-09-
01T00:00:00Z&limit=1000'

```

An empty `results` list means nothing has changed. Remember to URL-encode a `+` in a timezone offset as `%2B` , or use `Z` .

`since` reports new and changed entries. It does not report deleted ones. To detect deletions, compare `count` with your local total from time to time, and run a full download if they differ.

5.4 Resolve identifiers you already hold

By UUID: resolve up to 1000 at once. The response holds all of them on one page:

```

curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/rite-names/?uuids=0d6b1e87-2887-5b4c-b464-
a14d78895b30,5dfcff1d-ed9e-59b3-be0c-8b6296262474&fields=name'

```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "slug": "rite-names",
  "model": "indexerapp.RiteNames",
  "count": 2,
  "limit": 1000,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "0d6b1e87-2887-5b4c-b464-a14d78895b30",
      "id": 1,
      "name": "apostoli plures"
    },
    {
      "uuid": "5dfcff1d-ed9e-59b3-be0c-8b6296262474",
      "id": 2,
      "name": "de omnibus confessoribus"
    }
  ]
}
```

By legacy numeric id: for systems still holding the old database's numbering. Ids with no matching entry are listed in `unresolved_legacy_ids`:

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/formulas/?
legacy_ids=1,2,999999&fields=co_no,text'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "slug": "formulas",
  "model": "indexerapp.Formulas",
  "count": 1,
  "limit": 1000,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "44064845-17c3-5f94-9756-93588ebfa2e0",
      "id": 1,
      "legacy_id": 1,
      "co_no": "2",
      "text": "Exorcizo te creatura salis per Deum vivum per Deum verum et ..."
    }
  ],
  "unresolved_legacy_ids": [
```

```
2,  
999999  
]  
}
```

By name: use `search`. It is a substring match, so pick the exact entry from the results:

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/rite-names/?  
search=pro%20rege&fields=name,section_uuid&limit=3'
```

RESPONSE **HTTP 200**

```
{  
  "api_version": "v1",  
  "slug": "rite-names",  
  "model": "indexerapp.RiteNames",  
  "count": 2,  
  "limit": 3,  
  "offset": 0,  
  "next_offset": null,  
  "results": [  
    {  
      "uuid": "fc11931f-adf4-56d9-9f01-4cacf08ddcad",  
      "id": 3878,  
      "name": "pro rege",  
      "section_uuid": null  
    },  
    {  
      "uuid": "36f734e4-976f-5bea-baa5-d7c8b843e0fa",  
      "id": 4149,  
      "name": "LXXIV. Item alia missa pro rege",  
      "section_uuid": "a021510e-7d2c-5de5-ba7d-556be5fcbcff"  
    }  
  ]  
}
```

5.5 Import a manuscript with its content

Goal: an external tool has transcribed a manuscript and wants to publish it on MPL Limbo. Every response below is real, produced by the same API code running on a test database. That is why `site_name` reads `eCatalogus` and the UUIDs belong to test records.

1 Check the credentials

```
curl -u akowalska:PASSWORD https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/whoami/
```

Continue only if the response has `"can_import": true`. See section 2.3.

2 Check whether the manuscript already exists

Look it up by your own identifier:


```
curl 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/?foreign_id=ritus-2291'
```

If `count` is 1, reuse its `uuid` and go to step 4. Here is what the response looks like once the manuscript exists:

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "count": 1,
  "limit": 100,
  "offset": 0,
  "next_offset": null,
  "results": [
    {
      "uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
      "name": "Graduale Cracoviense",
      "rism_id": "PL-Kk 12",
      "foreign_id": "ritus-2291",
      "shelf_mark": "MS 12",
      "contemporary_repository_place_label": "The National Library",
      "dating_label": "XIV",
      "content_count": 0,
      "entry_date": "2026-09-28T12:13:36.191153+00:00"
    }
  ]
}
```

3

Create the manuscript

First confirm that the vocabulary values you want to use exist. For example:

```
curl 'https://ecatalogus.ispan.pl/api/v1/dictionaries/time-reference/?search=XIV&fields=time_description'
```

Then create the manuscript:

```
curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/ \
-H 'Content-Type: application/json' \
-d '{"name": "Graduale Cracoviense", "foreign_id": "ritus-2291", "rism_id": "PL-Kk 12",
    "shelf_mark": "MS 12", "contemporary_repository_place": "Biblioteka Narodowa",
    "dating": "XIV", "main_script": "Textualis"}'
```

RESPONSE **HTTP 201**

```
{
  "uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "name": "Graduale Cracoviense",
```

```

    "foreign_id": "ritus-2291",
    "content_count": 0,
    "entry_date": "2026-09-28T12:13:36.191153+00:00"
  }

```

Keep `uuid`. The examples below use `$MS` for it.

4

Make sure it has no content yet

```

curl https://limbo.monumenta-poloniae-
liturgica.ispan.pl/api/v1/manuscripts/$MS/content/summary/

```

RESPONSE **HTTP 200**

```

{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "manuscript_name": "Graduale Cracoviense",
  "content_count": 0,
  "has_content": false,
  "sequence_in_ms_min": null,
  "sequence_in_ms_max": null,
  "last_modified": null
}

```

If `has_content` is true, stop, or deliberately use `mode: "replace"` (section 5.6).

5

Prepare the rows

Relations may be given by name. `formula_id` needs a UUID, which you find by its Corpus Oratorium number with `/dictionaries/formulas/?search=2046`. Save this as `content.json`:

```

{
  "items": [
    {
      "sequence_in_ms": 1,
      "where_in_ms_from": "5",
      "where_in_ms_to": "6",
      "rubric_name_from_ms": "Missa pro rege et exercitu eius",
      "rubric_id": "pro rege",
      "formula_id": "2f1a32fb-721b-5655-9865-1b10eea70a10",
      "function_id": "Collecta",
      "original_or_added": "ADDED"
    },
    {
      "sequence_in_ms": 2,
      "where_in_ms_from": "6",
      "function_id": "Secreta",
      "formula_text_from_ms": "Munera domine quaesumus oblata sanctifica"
    }
  ]
}

```

```
]
}
```

6 Validate with a dry run

Add `"dry_run": true` to the object and post it. Nothing is written:

```
curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-  
liturgica.ispan.pl/api/v1/manuscripts/$MS/content/bulk/ \  
-H 'Content-Type: application/json' -d @content-dry-run.json
```

RESPONSE **HTTP 200**

```
{  
  "api_version": "v1",  
  "site_name": "eCatalogus",  
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",  
  "manuscript_name": "Graduale Cracoviense",  
  "mode": "append",  
  "dry_run": true,  
  "received": 2,  
  "created": 0,  
  "deleted": 0,  
  "errors": [],  
  "detail": "Payload is valid. Nothing was written because dry_run was set."  
}
```

If there are problems, all of them come back at once. Here a rubric was confused with a function, a sequence number was written in words, a function name was misspelt and an unknown key was used:

```
{"items": [  
  {"rubric_id": "Ad complendum", "where_in_ms_from": "7"},  
  {"sequence_in_ms": "twelve", "function_id": "Colecta", "folio": "7v"}  
], "dry_run": true}
```

RESPONSE **HTTP 400**

```
{  
  "detail": "4 problem(s) found; nothing was imported.",  
  "errors": [  
    {  
      "row": 0,  
      "field": "rubric_id",  
      "value": "Ad complendum",  
      "error": "not_found",  
      "detail": "No RiteNames entry matches \"Ad complendum\" (matched against: name).  
Dictionary entries must exist before content referencing them is imported."  
    },  
    {  
      "row": 1,  
      "field": "folio",
```

```

    "value": "7v",
    "error": "unknown_field",
    "detail": "\"folio\" is not a recognised content field."
  },
  {
    "row": 1,
    "field": "sequence_in_ms",
    "value": "twelve",
    "error": "invalid_value",
    "detail": "expected a whole number, got \"twelve\""
  },
  {
    "row": 1,
    "field": "function_id",
    "value": "Colecta",
    "error": "not_found",
    "detail": "No ContentFunctions entry matches \"Colecta\" (matched against: name).
Dictionary entries must exist before content referencing them is imported."
  }
]
}

```

`row` is the zero-based position in `items`. Fix the rows and repeat the dry run until it passes.

7

Import

```

curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-
liturgica.ispan.pl/api/v1/manuscripts/$MS/content/bulk/ \
-H 'Content-Type: application/json' -d @content.json

```

RESPONSE HTTP 200

```

{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "manuscript_name": "Graduale Cracoviense",
  "mode": "append",
  "dry_run": false,
  "received": 2,
  "created": 2,
  "deleted": 0,
  "errors": [],
  "created_uuids": [
    "e667f9b1-3f41-4be1-8721-58104ffd986f",
    "f224a6fe-b1a3-45d9-9667-1693c4895259"
  ]
}

```

```
curl https://limbo.monumenta-poloniae-
liturgica.ispan.pl/api/v1/manuscripts/$MS/content/summary/
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "manuscript_name": "Graduale Cracoviense",
  "content_count": 2,
  "has_content": true,
  "sequence_in_ms_min": 1,
  "sequence_in_ms_max": 2,
  "last_modified": "2026-09-28T12:13:36.798469+00:00"
}
```

5.6 Correct and re-upload content

Append more rows. Post them with the default mode. A bare list works too. Rows without `sequence_in_ms` continue after the highest existing number:

```
curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-
liturgica.ispan.pl/api/v1/manuscripts/$MS/content/bulk/ \
-H 'Content-Type: application/json' \
-d '[{"function_id": "Ad complendum", "where_in_ms_from": "6v"}]'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "manuscript_name": "Graduale Cracoviense",
  "mode": "append",
  "dry_run": false,
  "received": 1,
  "created": 1,
  "deleted": 0,
  "errors": [],
  "created_uuids": [
    "426b85f9-adfb-4d3b-b04b-acdc17e6c530"
  ]
}
```

Replace everything. After corrections, upload the full corrected set with `"mode": "replace"`. The old rows are deleted and the new ones written in one transaction. If validation fails, the old content stays untouched:

```
curl -u akowalska:PASSWORD -X POST https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/$MS/content/bulk/ \
-H 'Content-Type: application/json' -d '{"mode": "replace", "items": [ ... ]}'
```

RESPONSE **HTTP 200**

```
{
  "api_version": "v1",
  "site_name": "eCatalogus",
  "manuscript_uuid": "74bd9697-1afa-496d-a2a7-2f946ce26bd9",
  "manuscript_name": "Graduale Cracoviense",
  "mode": "replace",
  "dry_run": false,
  "received": 2,
  "created": 2,
  "deleted": 3,
  "errors": [],
  "created_uuids": [
    "1c6b9ba6-f53a-40be-b63b-2dba6c08caba",
    "9b9e41f0-0d89-4d0c-abb4-df03a7e30b44"
  ]
}
```

Round trip. The output of `GET .../content/` can be posted back unchanged. `uuid`, `manuscript_uuid`, `entry_date` and every `*_label` are accepted and ignored. To edit content in bulk, export it, change it, and re-upload it with `replace`.

5.7 Upload from a web page (JavaScript)

This is the flow used by ritus-indexer. The user types their eCatalogus login into the page, which keeps it in memory only for the upload.

```
const BASE = 'https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1';

const authHeader = (user, password) => 'Basic ' + btoa(`${user}:${password}`);

async function confirmIdentity(auth) {
  const response = await fetch(`${BASE}/whoami/`, { headers: { Authorization: auth } });
  if (response.status === 401) throw new Error('Wrong eCatalogus username or password.');
```

```
  const who = await response.json();
  if (!who.can_import) throw new Error(
    `Account ${who.username} may not import. Ask the administrator to add it to api_importers.`);
  return who;
}

async function upload(manuscriptUuid, rows, auth) {
  // 1. Do not duplicate work (public read, no credentials needed).
  const summary = await (await fetch(
    `${BASE}/manuscripts/${manuscriptUuid}/content/summary/`)).json();
  if (summary.has_content) throw new Error(`Already has ${summary.content_count} rows.`);
```

```

const send = (body) => fetch(`${BASE}/manuscripts/${manuscriptUuid}/content/bulk/`, {
  method: 'POST',
  headers: { Authorization: auth, 'Content-Type': 'application/json' },
  body: JSON.stringify(body),
});

// 2. Validate everything first: one request, every problem at once.
const check = await send({ items: rows, dry_run: true });
if (!check.ok) {
  const { detail, errors = [] } = await check.json();
  throw new Error([detail, ...errors.map(e => `row ${e.row}: ${e.field}: ${e.detail}`)].join('\n'));
}

// 3. Import for real.
const result = await send({ items: rows });
if (!result.ok) throw new Error((await result.json()).detail);
return (await result.json()).created;
}

// Wiring it to a form:
const auth = authHeader(form.username.value, form.password.value);
const who = await confirmIdentity(auth);
status.textContent = `Signed in as ${who.display_name}. Uploading...`;
status.textContent = `Imported ${await upload(manuscriptUuid, rows, auth)} rows.`;

```

The page's origin must be on the instance's CORS allow-list (section 2.4).

5.8 Cite the data you used

CC BY-NC 4.0 requires you to credit the source and to keep your use non-commercial. The package export names the people who catalogued the manuscript and provides the exact wording:

RESPONSE **HTTP 200** *rights block of the Antiphonarium package*

```

{
  "rights": {
    "license": "CC-BY-NC-4.0",
    "license_name": "Creative Commons Attribution-NonCommercial 4.0 International",
    "license_url": "https://creativecommons.org/licenses/by-nc/4.0/",
    "copyright": "Copyright (c) 2024–2026 Instytut Sztuki Polskiej Akademii Nauk (PAN) – Polish Academy of Sciences.",
    "rights_holder": "Instytut Sztuki Polskiej Akademii Nauk (PAN)",
    "rights_holder_url": "https://ispan.pl/",
    "required_statement": "Data from eCatalogus, Instytut Sztuki Polskiej Akademii Nauk (PAN). Used under CC BY-NC 4.0; not for commercial use.",
    "attribution": "\"Antiphonarium\", contributed by Paulina Pludra-Żuk. Data from eCatalogus, Instytut Sztuki Polskiej Akademii Nauk (PAN). Used under CC BY-NC 4.0; not for commercial use.",
    "accessed": "2026-09-28T12:02:44.200240+00:00",
    "source": "https://limbo.monumenta-poloniae-liturgica.ispan.pl/api/v1/manuscripts/28d56828-30eb-

```

```

45fe-9034-d3d7d6e88d12/package/",
  "recommended_citation": "Paulina Pludra-Żuk. \"Antiphonarium\". MPL Limbo. Instytut Sztuki
Polskiej Akademii Nauk (PAN). accessed 2026-09-28. https://limbo.monumenta-poloniae-
liturgica.ispan.pl/api/v1/manuscripts/28d56828-30eb-45fe-9034-d3d7d6e88d12/package/. Licensed under
CC-BY-NC-4.0.",
  "contributors": [
    {
      "uuid": "f34f11c0-9d7d-5041-ba01-cd9c99e3a167",
      "name": "Paulina Pludra-Żuk",
      "initials": "PPŻ",
      "affiliation": null,
      "url": null
    }
  ]
}

```

- Show `attribution` wherever you display the data.
- Use `recommended_citation` in publications.
- Do not use the data, or anything derived from it, for commercial purposes without permission from the rights holder.
- Keep `source` and `accessed` with any saved copy. The licence header is lost once a response is saved to a file.

6. Field reference

6.1 Content row

The same keys are used for reading (GET `.../content/`) and writing (POST `.../content/bulk/`). Every key is optional when writing.

Plain fields

Key	Type	Meaning
<code>sequence_in_ms</code>	integer	Position in the manuscript. Assigned automatically after the highest existing one if omitted.
<code>rubric_sequence_in_the_MS</code>	integer	Position of the rubric within the manuscript
<code>where_in_ms_from</code> , <code>where_in_ms_to</code>	text	Folio or page range, e.g. "12v"
<code>digital_page_number</code>	integer	Page in the digital facsimile
<code>rubric_name_from_ms</code>	text	Rubric as written in the manuscript
<code>subrubric_name_from_ms</code>	text	Sub-rubric as written
<code>formula_text_from_ms</code>	text	Text as it stands in the manuscript
<code>original_or_added</code>	text	ORIGINAL or ADDED (a later addition)
<code>proper_texts</code>	boolean	true / false ; "yes" , "no" , 1 , 0 are also accepted
<code>biblical_reference</code>	text	e.g. "Ps 50,3"
<code>reference_to_other_items</code>	text	Cross-reference, e.g. "ut supra"
<code>similarity_by_user</code>	text	Editor's similarity judgement
<code>edition_subindex</code>	text	Sub-index within the edition entry
<code>comments</code>	text	Free comment

Relations

When writing, give either the UUID or a name, matched case-insensitively against the listed columns. The name must match an entry exactly (no partial matches), and unknown values are errors. When reading, each key holds the UUID and is followed by a `*_label` key (e.g. `rubric_id` → `rubric_label` , `layer` → `layer_label`).

Key	Vocabulary (slug)	Name accepted from
<code>rubric_id</code>	rite-names	<code>name</code>
<code>formula_id</code>	formulas	UUID only. Look it up by <code>co_no</code> with <code>?search=</code> .
<code>liturgical_genre_id</code>	liturgical-genres	<code>title</code>
<code>section_id</code> , <code>subsection_id</code>	sections	<code>name</code>

function_id , subfunction_id	content-functions	name (e.g. Collecta, Secreta, Ad complendum)
layer	layers	short_name , then name
mass_hour	mass-hours	short_name , then name
genre	genres	short_name , then name
season_month	seasons-and-months	short_name , then name
week	weeks	short_name , then name
day	days	short_name , then name
text_standarization	text-standarization	standard_incipit
contributor_id	contributors	initials
quire_id	the manuscript's quires	UUID only
music_notation_id	music-notation-names	UUID of the manuscript's notation record, or a notation name . A name resolves to this manuscript's own notation record, which must already exist.
edition_index	edition content	UUID, or "<bibliography shortname> c.<sequence>"

Rubric or function? *Collecta*, *Secreta* and *Ad complendum* are liturgical **functions** (function_id). A **rubric** (rubric_id) names the occasion, such as *pro rege* or *de omnibus martyribus*. Putting a function in rubric_id is the most common import error.

6.2 Manuscript (for POST /manuscripts/)

Key	Type	Meaning
name	text	Required. Display name
foreign_id	text	Your system's identifier. Filter on it with ? foreign_id= .
rism_id , shelf_mark , usuarium_shelfmark , common_name	text	Identification
contemporary_repository_place	relation	places: repository_today_local_language or repository_today_eng
place_of_origin , binding_place	relation	places: repository_today_eng or repository_today_local_language
dating , binding_date	relation	time-reference: time_description (e.g. XIV , XI med)
main_script	relation	script-names: name

<code>how_many_columns_mostly</code> , <code>lines_per_page_usually</code> , <code>how_many_quires</code>	integer	Layout and codicology
<code>decorated</code> , <code>music_notation</code> , <code>display_as_main</code>	boolean	<code>display_as_main</code> defaults to true for API-created manuscripts
<code>foliation_or_pagination</code>	text	FOLIATION or PAGINATION
<code>form_of_an_item</code>	text	Form code, e.g. C (codex)
<code>liturgical_genre_comment</code> , <code>dating_comment</code> , <code>place_of_origin_comment</code> , <code>quires_comment</code> , <code>decoration_comments</code> , <code>music_notation_comments</code> , <code>general_comment</code>	text	Free comments
<code>links</code> , <code>additional_url</code> , <code>iiif_manifest_url</code> , <code>pdf_url</code>	text	Links to facsimiles and other resources
<code>connected_ms</code> , <code>where_in_connected_ms</code>	text	Link to a related manuscript

6.3 Vocabularies

Slugs for `/api/v1/dictionaries/{slug}/` and the columns that `?search=` matches. GET `/api/v1/dictionaries/` gives the current number of entries in each.

Slug	Name	Searched columns
<code>binding-components</code>	Binding Components	<code>name</code>
<code>binding-decoration-types</code>	Binding Decoration Types	<code>name</code>
<code>binding-materials</code>	Binding Materials	<code>name</code>
<code>binding-styles</code>	Binding Styles	<code>name</code>
<code>binding-types</code>	Binding Types	<code>name</code>
<code>ceremonies</code>	Ceremonies	<code>name</code>
<code>characteristics</code>	Characteristics	<code>name</code>
<code>colours</code>	Colours	<code>name</code>
<code>content-functions</code>	Content functions	<code>name</code>
<code>contributors</code>	Contributors	<code>initials</code> , <code>last_name</code> , <code>first_name</code>
<code>days</code>	Days	<code>short_name</code> , <code>name</code>
<code>decoration-techniques</code>	Decoration Techniques	<code>name</code>
<code>decoration-types</code>	Decoration Types	<code>name</code>
<code>feast-ranks</code>	Feast Ranks	<code>name</code>
<code>formulas</code>	Formulas	<code>co_no</code> , <code>text</code>
<code>genres</code>	Genres	<code>short_name</code> , <code>name</code>
<code>layers</code>	Layers	<code>short_name</code> , <code>name</code>
<code>liturgical-genres</code>	Liturgical Genres	<code>title</code>

mass-hours	Mass/Hours	short_name, name
music-notation-names	Music Notation Names	name
places	Places	repository_today_eng, repository_today_local_language
rite-names	Rite Names Standardized	name, english_translation
script-names	Script Names	name
seasons-and-months	Seasons/Months	short_name, name
sections	Sections	name
subjects	Subjects	name
text-standarization	text standarizations	standard_incipit, cantus_id, usu_id
time-reference	Time References	time_description
topics	Topics	name
traditions	Traditions	name
types	Types	short_name, name
weeks	Weeks	short_name, name

A sample entry of `places` , showing a vocabulary with many columns:

RESPONSE

```
{
  "uuid": "2ac35912-7e99-5ef0-bd8b-aed4e08a4d25",
  "longitude": 21.0067249,
  "latitude": 52.2319581,
  "place_type": null,
  "country_today_eng": "Poland",
  "region_today_eng": null,
  "city_today_eng": null,
  "repository_today_eng": null,
  "country_today_local_language": null,
  "region_today_local_language": null,
  "city_today_local_language": null,
  "repository_today_local_language": "Biblioteka Narodowa",
  "country_historic_eng": null,
  "region_historic_eng": null,
  "city_historic_eng": null,
  "repository_historic_eng": null,
  "country_historic_local_language": null,
  "region_historic_local_language": null,
  "city_historic_local_language": null,
  "repository_historic_local_language": null,
  "country_historic_latin": null,
  "region_historic_latin": null,
  "city_historic_latin": null,
  "repository_historic_latin": null,
}
```

```
"entry_date": "2026-04-27T09:21:04.373058+00:00"  
}
```

7. Error reference

HTTP status codes

Status	When	Body
200	Success, including a passed dry run and a completed bulk import	payload
201	Manuscript created	the new manuscript
400	Invalid query or payload	detail , plus errors on write endpoints
401	No credentials on a write, or a wrong password	detail
403	Valid account without import rights	detail
404	Unknown manuscript UUID or vocabulary slug	detail
429	Anonymous rate limit (15 per minute) exceeded	detail , Retry-After header

Row error codes (`errors[].error`)

Code	Meaning	Fix
not_found	No vocabulary entry matches the UUID or name	Check the spelling against <code>/dictionaries/{slug}/?search=</code> , or ask the editors to add the term
invalid_value	Wrong type, e.g. text in an integer field	Send a whole number or a boolean
unknown_field	Unrecognised key	Correct the key (section 6), or send <code>strict: false</code>
invalid_row	An item is not a JSON object	Send objects in <code>items</code>
required	A required field is missing or blank (manuscript <code>name</code>)	Provide it
too_many_errors	Reporting stopped after 200 problems	Fix the reported ones and validate again

Real error responses

Write without credentials

RESPONSE **HTTP 401**

```
{
  "detail": "Authentication credentials were not provided."
}
```

Write with an account that lacks import rights

RESPONSE **HTTP 403** *bulk import*

```
{
  "detail": "You do not have permission to access this endpoint."
}
```

```
}
```

RESPONSE **HTTP 403** *manuscript creation*

```
{
  "detail": "You do not have permission to modify catalogue data."
}
```

Empty items, unknown mode

RESPONSE **HTTP 400**

```
{
  "detail": "\"items\" is empty – nothing to import.",
  "errors": []
}
```

RESPONSE **HTTP 400**

```
{
  "detail": "Unknown mode \"overwrite\". Use \"append\" or \"replace\".",
  "errors": []
}
```

Dictionary queries

RESPONSE **HTTP 400** *?q=abc (unknown parameter)*

```
{
  "detail": "Unknown query parameter(s): q. This endpoint accepts: fields, format, legacy_ids, limit, offset, search, since, uuids."
}
```

RESPONSE **HTTP 400** *?uuids=abc*

```
{
  "detail": "\"uuids\" contains 1 value(s) that are not UUIDs: \"abc\""
}
```

RESPONSE **HTTP 400** *?since=yesterday*

```
{
  "detail": "Could not parse \"since\" value \"yesterday\" as an ISO-8601 timestamp."
}
```

RESPONSE **HTTP 404** */dictionaries/nope/*

```
{
  "detail": "Unknown dictionary \"nope\"."
}
```

8. Troubleshooting and FAQ

A search returns nothing, although the term exists.

Search only looks at the vocabulary's name columns (section 6.3), and it is a substring match. It does not ignore diacritics or spelling variants: *complendum* finds *Ad complendum* in `content-functions`, but not in `rite-names`. Make sure you are searching the right vocabulary.

My import fails with `not_found`, but the search finds the entry.

Import names must match an entry exactly (case does not matter), while search matches substrings. Send the exact `name` from the search results, or send its `uuid`.

Can I create vocabulary entries through the API?

No, this is deliberate. Vocabularies are curated in eCatalogus and replicated to all instances. Ask the editors to add the term, then retry.

The browser shows a CORS error.

Your page's origin is not on the instance's allow-list. Ask the administrator to add it to `API_INTEGRATION_ORIGINS` (section 2.4). Also make sure you are not sending `credentials: 'include'`.

I get 401 from a browser, but the same credentials work with curl.

```
btoa() fails on characters outside Latin-1. Encode the credentials as UTF-8 first:  
btoa(String.fromCharCode(...new TextEncoder().encode(`${user}:${pass}`)))
```

Does a failed import leave partial data behind?

No. Validation covers the whole payload first, and the write is a single transaction.

A large import times out on my side.

The server does not cut long imports short. Raise your client timeout (about one second per hundred rows, plus a margin), or split the upload into several `append` requests.

Why do the manuscript UUIDs differ between instances?

Each instance holds its own manuscripts. Only vocabularies are shared. Store the instance's base URL together with each manuscript UUID.

I get 429 Too Many Requests.

Anonymous callers may make 15 requests per minute. Wait for the number of seconds in the `Retry-After` header, or send the credentials of any eCatalogus account: authenticated requests are never throttled.

How do I get the images of a manuscript?

Take the `url` of each entry in the package's `media_files` and download it directly. Images are served as ordinary files by the web server.

Appendix: a small Python client

This client covers the whole import flow of section 5.5. Credentials come from the environment.

```
import os
import requests

class ECatalogus:
    def __init__(self, base_url, user=None, password=None, timeout=600):
        self.base = base_url.rstrip("/") + "/api/v1"
        self.session = requests.Session()
        if user:
            self.session.auth = (user, password)
        self.timeout = timeout

    def _get(self, path, **params):
        response = self.session.get(self.base + path, params=params, timeout=self.timeout)
        response.raise_for_status()
        return response.json()

    def _post(self, path, body):
        response = self.session.post(self.base + path, json=body, timeout=self.timeout)
        if response.status_code == 400:
            raise ValueError(response.json())          # detail + every row error
        response.raise_for_status()
        return response.json()

    def whoami(self):
        return self._get("/whoami/")

    def find_by_foreign_id(self, foreign_id):
        results = self._get("/manuscripts/", foreign_id=foreign_id)["results"]
        return results[0] if results else None

    def create_manuscript(self, **fields):
        return self._post("/manuscripts/", fields)

    def summary(self, ms):
        return self._get(f"/manuscripts/{ms}/content/summary/")

    def import_content(self, ms, items, mode="append", dry_run=False):
        return self._post(f"/manuscripts/{ms}/content/bulk/",
                          {"items": items, "mode": mode, "dry_run": dry_run})

    def iter_dictionary(self, slug, **params):
        offset = 0
        while offset is not None:
            page = self._get(f"/dictionaries/{slug}/", limit=1000, offset=offset, **params)
```

```

        yield from page["results"]
        offset = page["next_offset"]

api = ECatalogus("https://limbo.monumenta-poloniae-liturgica.ispan.pl",
                 os.environ["ECAT_USER"], os.environ["ECAT_PASSWORD"])

assert api.whoami()["can_import"], "ask the administrator to add you to api_importers"

ms = api.find_by_foreign_id("ritus-2291") or api.create_manuscript(
    name="Graduale Cracoviense", foreign_id="ritus-2291", dating="XIV")

items = [{"sequence_in_ms": 1, "function_id": "Collecta", "where_in_ms_from": "5"}]
if not api.summary(ms["uuid"])["has_content"]:
    api.import_content(ms["uuid"], items, dry_run=True)          # raises on any problem
    print(api.import_content(ms["uuid"], items)["created"], "rows imported")

```

The interactive reference at </api/schema/swagger/> on every instance lists the same endpoints, and lets you try them. Use *Authorize* to enter credentials for the write endpoints.